

Reviewer Pack — Minimal Order of p-Adic L-Functions and DPLL Proof Length

Entry 886f010403ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 23:08:40 UTC

Minimal Order of p-Adic L-Functions and DPLL Proof Length

Entry ID: 886f010403ac **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 23:08:40 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (L-Function Theory) **Field B** (complexity object): DPLL Search Trees

Statement:

The minimal order of a p-adic L-function associated with a Boolean satisfiability problem's clause set φ is linearly correlated with the length of the shortest DPLL proof for φ , such that $\text{ord}_L(\varphi) = \Theta(l(\varphi))$ where ord_L denotes the order of vanishing at $s=0$ of the L-function and l denotes the length of the DPLL proof.

Rationale (proposer's reasoning):

p-adic L-functions encode arithmetic information about the structure of number fields, which could potentially reveal intricate aspects of Boolean satisfiability problems. This conjecture aims to explore a potential bridge between the algebraic geometry of p-adic numbers and the algorithmic complexity of logic solving.

Taxonomy category: ArithmeticHierarchy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7545d152bf1190e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal order of p-adic L-functions and the length of DPLL proofs for a set of 30 random CNF formulas exceeds 0.7, with no seed producing a correlation coefficient below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        unit_clauses = [c[0] for c in cnf if len(c) == 1]
        pure_literals = {}

        for literal in set([abs(lit) for lit in sum(cnf, [])]):
            pos_count = sum(1 for clause in cnf if literal in clause)
            neg_count = sum(1 for clause in cnf if -literal in clause)
            if pos_count == 0:
                pure_literals[literal] = True
            elif neg_count == 0:
                pure_literals[-literal] = True

        if unit_clauses:
            literal = unit_clauses[0]
            new_assignment[literal] = True
            cnf = [c for c in cnf if literal not in c and -literal not in c]
            return dpll(cnf, new_assignment)

        if pure_literals:
            literal = next(iter(pure_literals))
            new_assignment[literal] = True
```

```

        cnf = [c for c in cnf if literal not in c and -literal not in c]
        return dpll(cnf, new_assignment)

    literal = random.choice(sum(cnf, []))
    new_assignment[literal] = True
    cnf_true = [c for c in cnf if literal in c]
    cnf_false = [c for c in cnf if -literal in c]
    return dpll(cnf_true, new_assignment) or dpll(cnf_false, {**new_assignment, -literal: False})

def p_adic_l_function_order(cnf):
    # Simplified approximation of L-function order
    return len(cnf)

n = random.randint(5, 40)
m = random.randint(n, 2 * n)
cnf = generate_cnf(n, m)
l_phi = p_adic_l_function_order(cnf)
phi_assignment = {}
if dpll(cnf, phi_assignment):
    l_dpll = len(phi_assignment)
else:
    l_dpll = float('inf')

return {
    "metric_name": "correlation_coefficient",
    "metric_value": (l_phi - l_dpll) / max(l_phi, l_dpll),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs((l_phi - l_dpll) / max(l_phi, l_dpll)) < 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31, 100))
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient<0.5' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data_or_budget_exceeded")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7ef12fc.py", line 88, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7ef12fc.py", line 69, in run_trial
    if dpll(cnf, phi_assignment):
           ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7ef12fc.py", line 50, in dpll
    new_assignment[literal] = True
    ^^^^^^^^^^^^^^^^^
NameError: name 'new_assignment' is not defined. Did you mean: 'assignment'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing any data. As the critic challenged the conjecture and the pre-registered support condition was not unambiguously met, the verdict is INCONCLUSIVE. | next: Review the code for the missing 'new_assignment' variable and fix the error before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11815
2	preregistration	ollama_remote	glm4:latest	0	0	14740
3	novelty	ollama_remote	glm4:latest	0	0	8745
4	novelty	ollama_remote	glm4:latest	0	0	9038
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15520
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26565
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21731
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10989
9	judge	ollama_remote	glm4:latest	0	0	13288

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132430 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in <research/audit/886f010403ac.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/886f010403ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/886f010403ac.tar.gz` (if generated)